포트폴리오 2026 / 개발자

실제 주문이 들어오는 **자동 처리 시스템을 설계하고** 멈추지 않게 직접 운영했습니다.

온라인 판매 주문 처리 서버를 설계해 첫 코드 5일 만에 판매를 시작했고, 판매 6일째 하루 1,378건을 처리했습니다. 결과가 불명확한 주문은 다시 보내지 않고 보류한 뒤 거래 이력과 맞춰 확정하게 했고, 돈이 나가는 일과 AI 에이전트에게 맡길 일을 나눠 한도를 정했습니다.

현장실습에서는 고칠 수 없는 출입 시스템의 기록만 읽어 근무 상태를 판정하는 연동을 회사 서버에 배포했고, 3인 팀장으로 공공기관 복지 챗봇을 RAG로 다시 만들었습니다.



숫자로 보면

1,378건
하루 최대 주문 · 판매 6일째

5일
첫 코드에서 판매 시작까지

30분
출입 기록 → 근무 상태 판정 주기

1,809건
복지사업 · 매일 새벽 자동 반영

수상

2025 컴퓨터인공지능학부 작품경진대회 은상
2025 캡스톤디자인 우수상
2025 전북특별자치도사회서비스원 성과보고회 표창

배지혁

전북대학교 컴퓨터공학부 / qowlgur121@gmail.com / github.com/qowlgur121 / qowlgur121.github.io

소개

학력 · 현장실습 · 기술

학력

전북대학교	컴퓨터공학부 공학사 · 2022.03 ~ 2026.02
성적	전체 3.71/4.5 · 전공 3.83/4.5 3학년 2학기 4.50 (학과 1등)

현장실습

(주)텔로스	백엔드 개발 현장실습 · 2025.07 · 회사 서버 배포

수상

2025.12	작품경진대회 은상 · 전북대 컴퓨터인공지능학부
2025.12	캡스톤디자인 우수상 · 전북대학교
2025.11	성과보고회 표창 · 전북특별자치도사회서비스원

자격증 · 활동

자격증	정보처리기사 · SQLD · ADsP
JAVICE	SW 창업동아리 · 2023.09 ~ 2025.12
ALPS	알고리즘 동아리 스터디 멘토 · 2022.03 ~ 2023.07

기술

Java	현장실습 · OpenContext · JeonjuEats 백엔드
Spring Boot	현장실습 출입 판정 배치 · OpenContext 검색 API
Python	주문 자동화 시스템 · 운영 스크립트
FastAPI	복지 AI 챗봇 백엔드 · SSE 스트리밍
PostgreSQL	pgvector 벡터 검색 · OpenContext 메타데이터
Docker	Docker Compose · Nginx로 서버 배포

제가 일하는 방식

조건부터 문장으로	"같은 사람이 연달아 여러 개를 사도 되는가"처럼 실패 조건을 먼저 정하고, 그 조건을 견디는 구조를 골랐습니다
고칠 수 없는 시스템 옆에 붙이기	출입 DB · 기관 홈페이지 · 사내 문서처럼 원본을 바꿀 수 없을 때, 읽기만 하는 연동으로 기능을 더했습니다
운영하면서 기준 고치기	감시 주기 · 처리 한도 · 알림 기준을 실제 주문을 보며 직접 정하고, 고친 상황은 테스트로 남겼습니다

01

주문 자동화 시스템과 온라인 스토어 운영

대표 프로젝트 · 개요와 구조

개인 프로젝트 · 기획부터 운영까지 직접

2026.09 ~ 2026.10

온라인 판매를 열기 전에 주문 처리 서버부터 설계했습니다. 결제 확인부터 발주·전달·정산 대조까지 이어지게 해 판매 6일째 하루 1,378건을 처리했습니다.

처음 정한 조건	서버 비용은 최소로 · 보내기 전에 상품 코드가 유효한지 확인 · 운영 부담은 제가 감당할 수 있게
Stack	Python · SQLite WAL · unittest 회귀 테스트
Infra	Oracle Cloud ARM VM · Ubuntu · systemd 타이머 · 전용 계정과 ProtectSystem=strict
연동	네이버 커머스 API · 외부 주문 API · 문자 발송 · Telegram 알림
역할 분리	주문 처리는 정해진 코드로만 · AI 점검 에이전트(15분)는 상태를 읽고 허용된 조치만

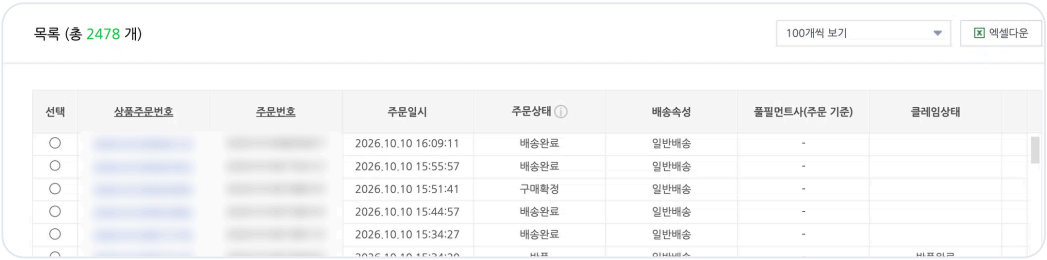
1,378건
하루 최대 주문 (10/07) · 9일 누적 2,469건

5일
첫 코드 → 판매 시작

20초
주문 조회 주기 · 백업 매일



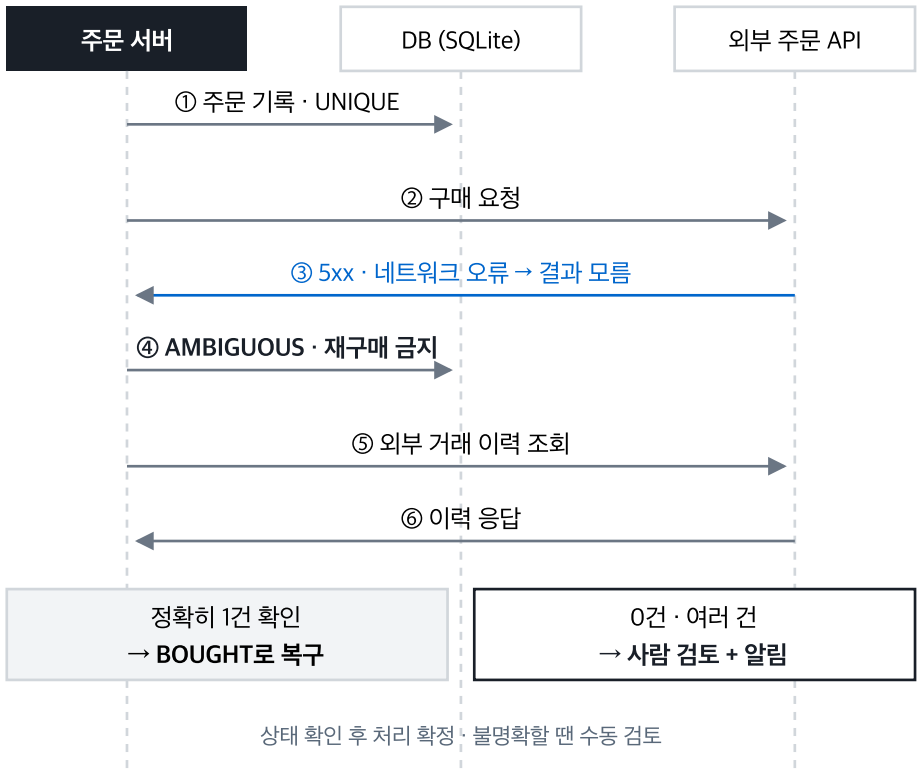
제가 나눈 시스템 구조. 회색 상자는 AI 에이전트 · 테두리 상자는 사람 판단



판매자센터 주문 목록 (주문번호는 가림)

응답이 끊긴 주문은 다시 보내지 않고, 보류한 뒤 거래 이력으로 확정하게 설계했습니다

문제 해결 1



구매 결과를 모를 때의 처리 순서

문제

판매 초기에 "같은 사람이 한 번에 여러 개를 사거나 연달아 사도 문제없어야 한다"를 조건으로 정했습니다. 그런데 외부 주문 API의 응답이 끊기면 처리됐는지 알 수 없었고, 그냥 재시도하면 같은 주문이 두 번 처리될 수 있었습니다.

설계 판단

자동 재시도	동일 요청이 중복 실행될 수 있음
즉시 실패	실제 처리 여부와 내부 상태가 어긋날 수 있음
선택한 방식	**상태 보류 → 거래 이력 확인 → 결과 확정**

결과가 불명확하면 **AMBIGUOUS**로 보류하고, 외부 거래 이력에서 일치하는 내역을 찾아 복구했습니다. 애매한 건은 사람 확인으로 넘겼습니다.

주문번호 **UNIQUE** 제약과 조건부 UPDATE로 동일 주문의 상태 전이를 보호했습니다.

결과

불명확한 주문을 재실행하지 않고 **대조 후 확정**하는 순서로 판매 6일째 하루 1,378건을 처리했습니다. 발송 직전에도 같은 1회용 상품 코드가 다른 주문에 나간 적이 있는지 한 번 더 확인하고, 겹치면 보내지 않고 멈추게 했습니다. 잘못 보내는 것보다 멈추는 편이 낫다고 판단했습니다.

서버 1대(Oracle Cloud ARM 1 OCPU · 6GB) · SQLite WAL · 20초 주기 조회
핵심 로직 예제 · github.com/qowlgur121/order-fulfillment-guard

오류 없이 조용히 틀린 장애 두 건을 기록으로 찾아 재발을 막았습니다

문제 해결 2



운영 중 문제를 찾아 재발을 막는 순서

문제

시험 환경에서는 통과했지만 실제 운영에서만 드러나는 문제가 있었습니다. 가장 위험한 것은 **오류 없이 조용히 틀리는 경우**였습니다.

성공인데 빈 답변	고객 문의 자동 답변이 오류 없이 등록됐지만 필드 이름 하나가 달라 내용이 비어 있었습니다 → 같은 날 복구하고, **등록 후 다시 읽어 내용을 확인**하게 바꿨습니다
DB 잠금	대량 작업이 외부 API를 39번 부르는 동안 DB 잠금을 쥐고 있어 주문 조회가 약 1분 멈췄습니다 → 누락 주문이 없는지 확인하고, **10건마다 나눠 저장**하고 잠금 대기 시간을 두었습니다

결과

두 건 모두 고객에게 피해가 커지기 전에 원인을 기록으로 확인하고, 같은 상황을 재현하는 테스트와 감시로 고정했습니다. "성공" 표시만 믿지 않고 **결과를 다시 읽어 확인하는 단계**를 운영 기준으로 삼았습니다.

처리 한도와 알림 기준을 정해, 주문이 몰려도 사고가 커지지 않게 했습니다

문제 해결 3

운영하면서 제가 정한 기준 4가지

① 하루 처리 상한
상한의 80%에서 미리 알림 · 막히면 원인을 그대로 표시
자동 처리가 폭주해도 피해가 상한 안에서 멈춤

② 조치가 필요한 것만 알림
정보성 이벤트는 기록만 · 요약은 하루 두 번
알림이 많으면 진짜 사고를 놓친다고 판단

③ 보냈다는 사실은 제3자 기록으로
내부 로그 대신 문자 발송 대행사의 발송 기록으로 확인
내가 만든 로그는 반박당할 수 있기 때문

④ 이상 판정은 비교 기준으로
최근 1시간 주문을 지난 3일 같은 시간대와 비교
15분 고정 점검은 대응이 늦다고 보고 기준을 바꿈

주문 처리를 멈추지 않게 하는 운영 기준

문제

주문이 몰리면 자동 처리가 계속 돈을 쓰며 폭주할 수 있었고, 반대로 알림이 쏟아지면 정말 손을 봐야 할 사고를 놓칠 수 있었습니다. 같은 경고 문장이 연달아 세 번 오는 일도 있었습니다.

해결

하루 처리 상한과 선결제 예치금·비축 수량의 상한을 제가 정하고, 상한에 가까워지면 미리 알리게 했습니다. 상한에 막혔을 때 "자동으로 다시 시도 중"이라고 표시하던 문구는 사실과 달라 원인을 그대로 쓰게 바꿨습니다.

알림은 조치가 필요한 것만 보내고, 고객에게 보냈다는 사실은 발송 대행사 기록으로 확인했습니다.

> **핵심 선택** · 자동화의 속도보다 사고가 났을 때 피해의 크기를 먼저 정함

결과

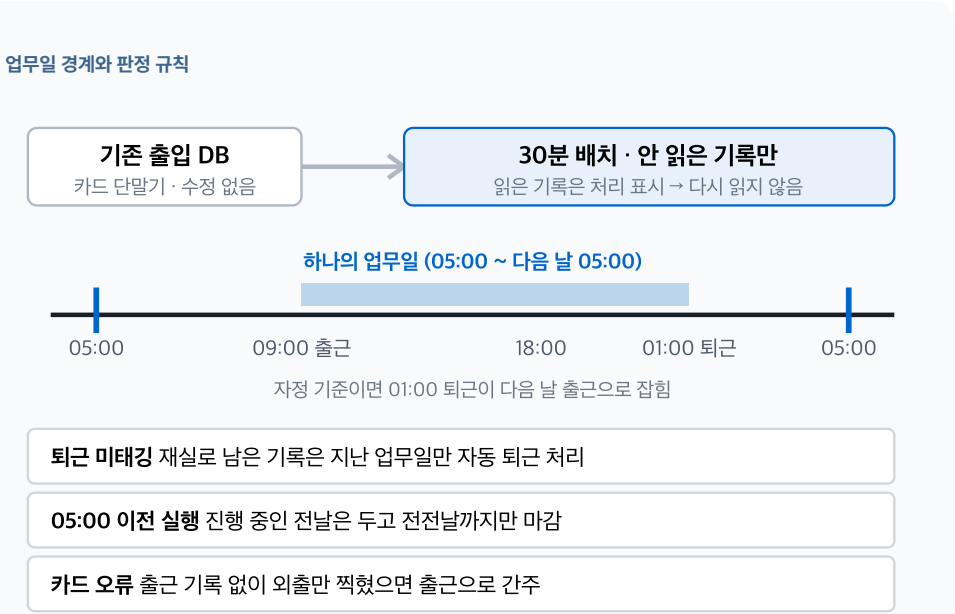
10월 5일 하루 상한에 걸려 주문 1건이 약 5분 늦어졌을 때, 원인 알림을 보고 상한을 250건으로 올려 바로 처리했습니다. 이후 정상 판매일 때만 상한이 늘어나는 조건도 정했습니다.

02

고칠 수 없는 출입 시스템의 기록만 읽어 직원 근무 상태를 판정했습니다

문제 해결 4 · 현장실습

(주)텔로스 · 백엔드 전담
2025.07 · 회사 서버 배포



프로젝트

직원 자리와 출근 · 재실 · 퇴근 상태를 한 화면에 보여 주는 **사내 자리배치도 웹**의 백엔드(Spring Boot)를 전담했습니다.

문제

회사에 있는 건 출입 카드 기록뿐이었고, 운영 중인 출입 시스템은 고칠 수 없었습니다. 퇴근 카드를 안 찍으면 계속 재실로 남고, 새벽 퇴근은 다음 날 출근처럼 보였습니다.

해결

운영 중인 시스템을 건드리지 않도록 원본 DB는 읽기만 하고 30분마다 새 기록만 가져오는 방식을 택했습니다. 하루를 **새벽 5시 기준**으로 나눠 야근 기록이 이틀로 쪼개지지 않게 했고, 퇴근 기록이 없으면 지난 업무일만 자동 퇴근 처리했습니다. 외출만 찍고 들어온 카드 오류는 출근으로 봤습니다.

결과

원본 시스템을 바꾸지 않고 카드 기록만으로 직원 상태를 보여 주게 됐고, 회사 서버에 배포했습니다.

Spring Boot 3.5 · Java 17 · SQL Server · 프론트엔드 선임과 2인 개발

같은 프로젝트에서 만든 것

좌석 배정 API(중복 배정 방지) · 직원 · 권한 관리 API · 공통 오류 형식(오류 코드 26개) · JWT 로그인(HttpOnly) · Swagger 문서 · Docker Compose 배포

다른 개발 경험

JeonjuEats · Spring Boot/Vue 기반 주문 서비스, AWS 배포
Calelog · 4인 팀 일정 기반 활동일지 자동화, 백엔드·CI/CD

배지혁 github.com/qowlgur121

공공 복지 AI 챗봇 고도화 · RAG

대표 프로젝트 · 개요와 구조

산학 프로젝트 · 3인 팀 팀장
2025.09.22 ~ 2025.12.05

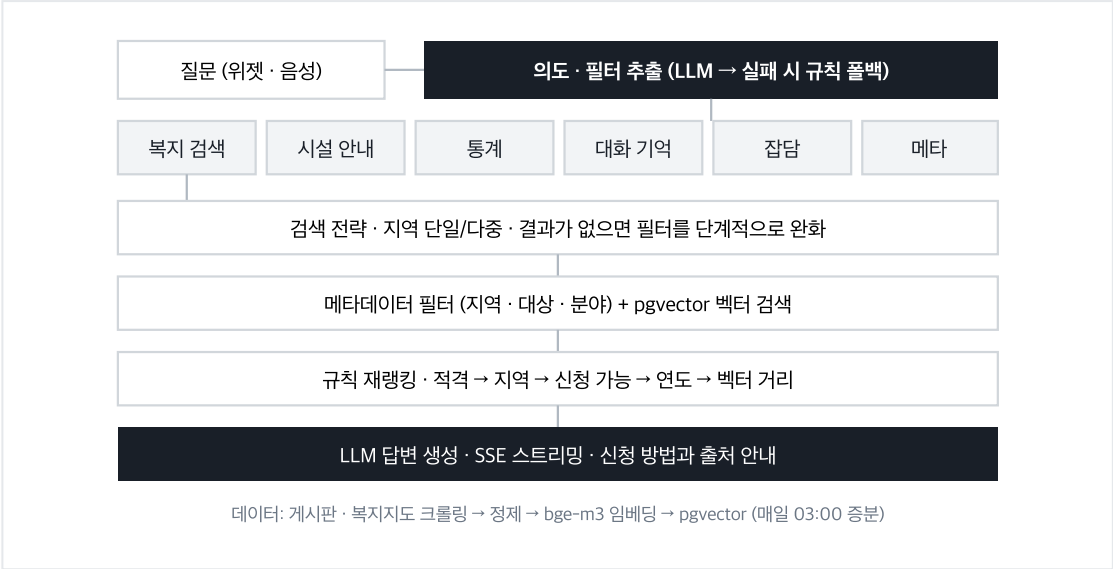
표현이 조금만 달라도 답을 못 찾던 기관 챗봇을 의도 판단과 RAG 검색으로 다시 만들었습니다. 팀장으로 기관과 요구사항을 맞추고 개발을 맡았습니다.

협력	전북특별자치도사회서비스원
처음 정한 조건	기관 원본 시스템은 고칠 수 없음 · 운영 담당자는 개발자가 아님 · 회원 개인정보는 받지 않음
Backend	FastAPI · SQLAlchemy · APScheduler · SSE · Docker Compose · 위젯 Custom Element
AI · 검색	LangChain 0.3 · bge-m3 임베딩 · PostgreSQL + pgvector · LLM 클라우드/로컬 선택
역할	**내 담당** 요구사항 협의 · 설계 · 백엔드 · 검색 · 관리자 · 위젯 · 발표 **팀원** 자료 조사 · 발표 자료 · 문서

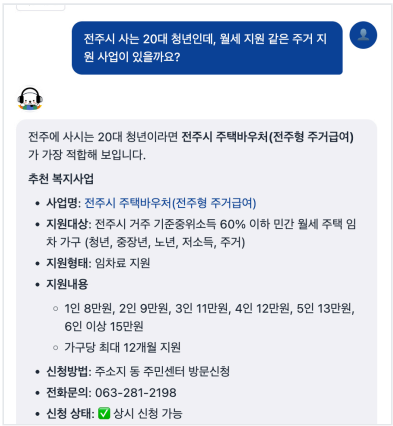
1,809건
복지사업 · 매일 03시 갱신

18,318곳
복지시설 위치 데이터

우수상
캡스톤 우수상 · 기관 표창



질문 하나가 답이 되기까지의 흐름





조건을 말하면 받을 수 있는 사업과 신청 방법 안내 / 증상을 말하면 가까운 의료기관을 거리순 안내

새 사업마다 재학습이 필요했던 챗봇을 매일 갱신되는 RAG 구조로 바꿨습니다

문제 해결 5



기존 방식과 새 방식 비교

문제

기존 챗봇은 파인튜닝 기반이라 학습한 Q&A 안에서만 답했고, 새 사업을 넣으려면 다시 학습해야 해서 신청 기간이 지난 사업을 안내하기도 했습니다.

해결

재학습 주기화 학습 비용이 들고 그 사이 정보는 계속 낡음

채택 **모델은 그대로 두고 매일 갱신되는 데이터를 찾아 답하는 RAG**

데이터 연동 API를 받을 수 없어 처음엔 기관 엑셀을 기준으로 삼으려 했지만, 공개 게시판을 직접 수집할 수 있음을 확인하고 **엑셀 의존을 걷어냈습니다.** 공공기관 사이트라 Spring으로 가면 전자정부 프레임워크까지 맞춰야 할 수 있어서 수집·검색 서버는 Python 하나로 묶었습니다.

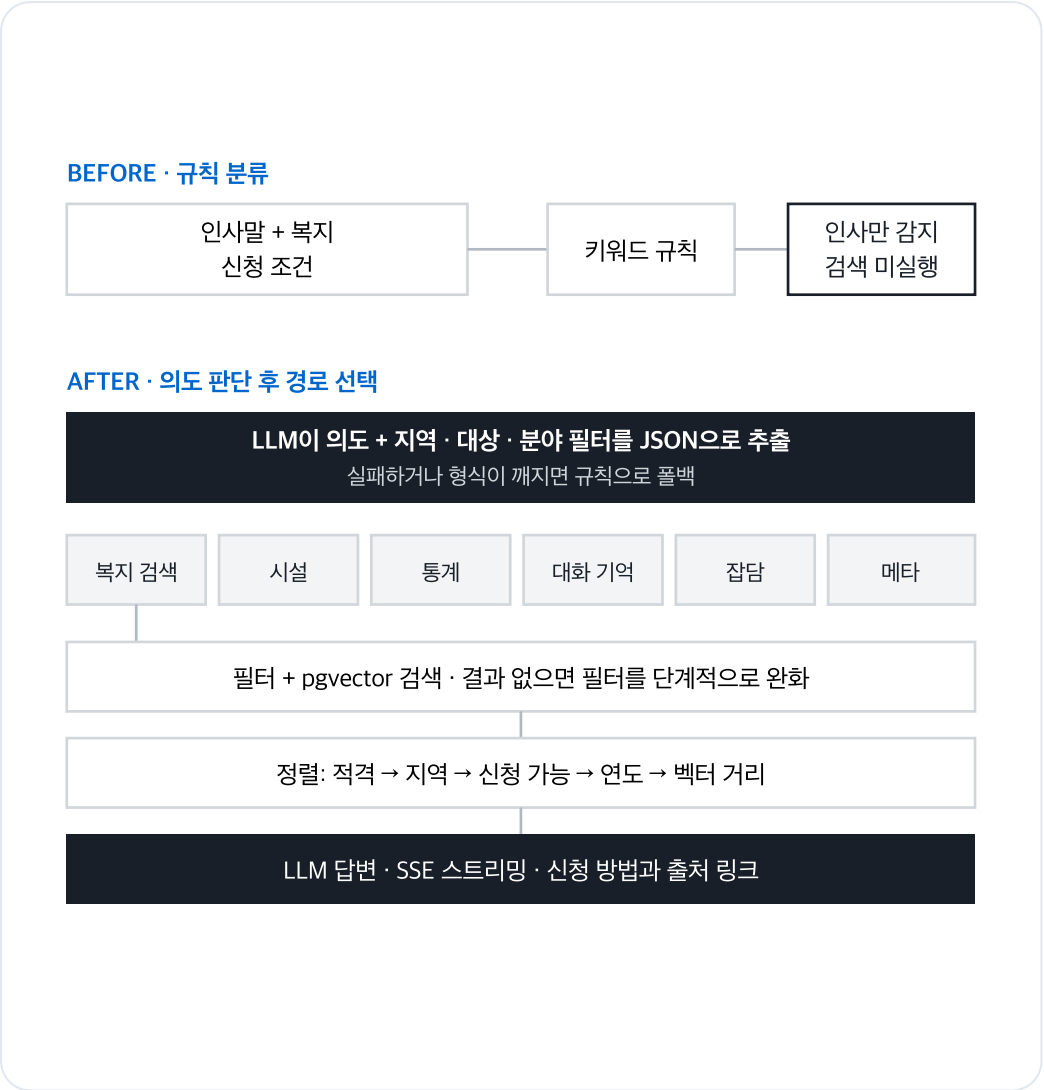
바뀐 사업만 매일 수집해 500자 이하로 나누고, 전용 벡터 DB 대신 PostgreSQL에 pgvector를 붙여 **기관이 관리할 시스템을 하나로** 줄였습니다.

결과

재학습 없이 복지사업 1,809건이 매일 새벽 반영되고, 찾은 원문과 공식 게시글 링크로 답합니다. 키워드 검색으로 0건이던 "치매" 질문에서 관련 사업 20건을 찾았습니다.

인사가 섞인 복지 질문도 검색하도록 의도 분류를 개선했습니다

문제 해결 6



질문 처리 흐름 개선 전후

문제

인사말과 신청 조건이 섞인 질문이 인사로 분류돼 검색을 건너뛰었습니다. 표현별 예외 규칙을 늘리는 것으로는 끝이 없었습니다.

설계 판단

규칙만 추가	표현이 달라질 때마다 예외 유지보수 필요
LLM 단독	응답 실패·형식 오류에 대한 별도 처리 필요
선택한 방식	**LLM으로 의도 추출 + 검증 후 규칙 폴백**

의도·지역·대상·분야를 JSON으로 뽑아 경로를 고르고, 오류가 나면 규칙 기반 분류로 넘어가게 했습니다. 검색 결과는 신청 가능한 사업부터 정렬했습니다.

답변은 **SSE 스트리밍**으로 전송해 생성 과정을 단계적으로 보여줬습니다.

결과

질문을 **6개 의도 경로**로 나눠 복지 검색으로 이었고, LLM 오류에도 규칙 기반으로 답하는 대체 경로를 두었습니다.

구현 · FastAPI · LangChain · JSON 구조화 · 규칙 폴백 · SSE

OpenContext · 사내 문서를 밖으로 보내지 않는 AI 검색

개요와 구조

3인 팀 · 오픈소스 개발자대회

2025.08 ~ 2025.12 · github.com/OpenContextAI/open-context

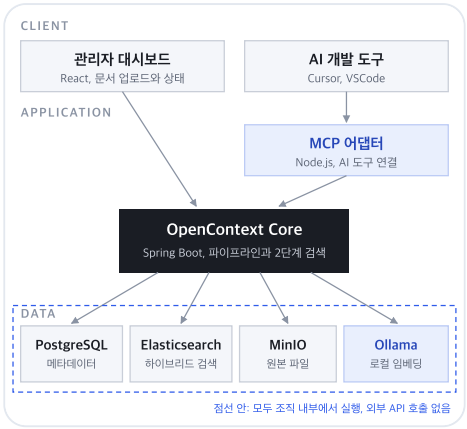
현장실습 때 외부 MCP 문서 서비스를 써 보니 효과가 컸지만 통제권이 밖에 있었습니다. 그래서 회사가 직접 통제하는 사내 버전을 제안해 팀원들과 만들었습니다.

Backend	Java 21 · Spring Boot 3.3 · QueryDSL · Flyway · LangChain4j
검색	Elasticsearch 8.11 · Ollama bge-m3 로컬 임베딩 · MinIO · PostgreSQL 16
연동	MCP (TypeScript SDK) · Cursor · VS Code
Admin	React 19 · Vite · TanStack Query · Zustand · Tailwind
역할	**내 담당** 제안 · 하이브리드 검색 엔진 · 2단계 검색 API · DB 모델링과 테스트 · 관리자 화면 · Docker 배포 · 보안 패치 **팀원** 문서 수집 파이프라인 · MCP 어댑터 · 공통 에러 처리

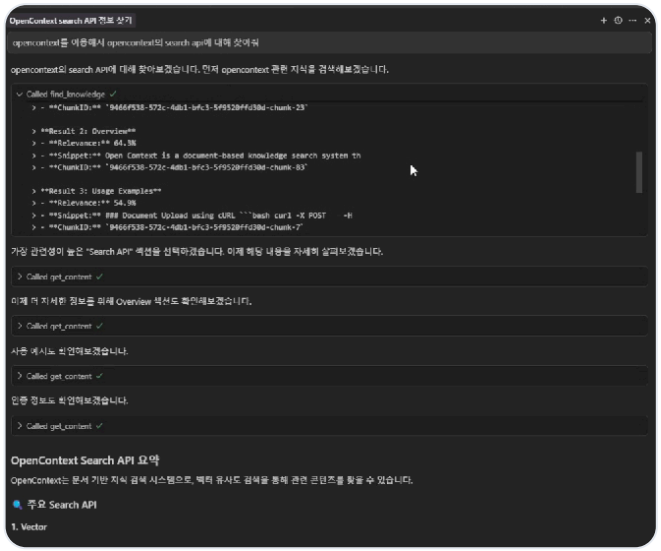
은상
학부 작품경진대회

로컬 처리
문서 처리 · 임베딩을 조직 안에서

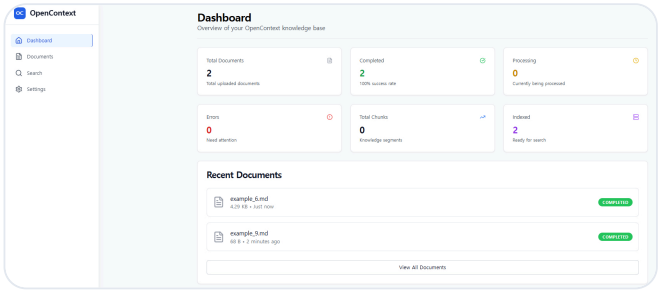
2단계
고르고 원문만 가져오기



전체 구조. 점선 안은 모두 조직 내부에서 실행



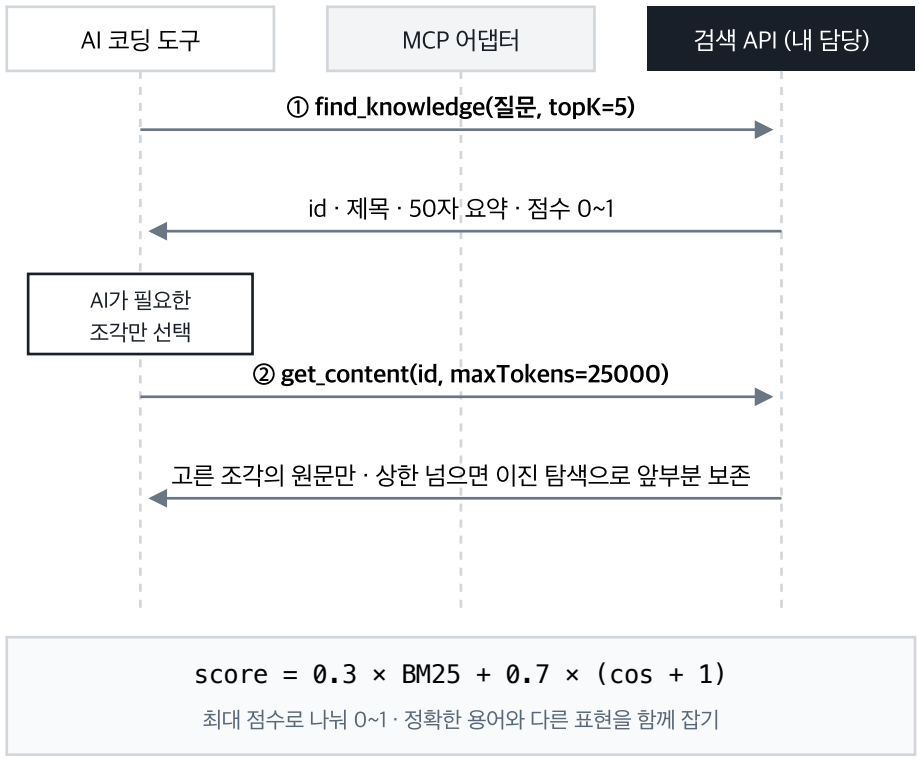
AI 도구가 find_knowledge로 고르고 get_content로 원문만 가져옴



제가 만든 관리자 화면 · 문서 처리 상태 확인

AI에 필요한 문서만 전달하는 2단계 검색 API를 설계했습니다

문제 해결 7



2단계 검색 호출 순서와 점수 계산

문제

검색한 원문을 한 번에 다 넘기면 컨텍스트가 커지고 AI가 쓸 자료를 고르기 어려웠습니다.

해결

find_knowledge에서 제목·요약·관련도 목록을 먼저 반환하고, AI가 선택한 문서만 **get_content**로 원문을 가져가도록 나눴습니다.

검색은 BM25 키워드 점수와 벡터 유사도를 합치고, 순위와 함께 관련도 점수를 돌려줬습니다.

> **핵심 선택** · 검색 결과 탐색과 원문 읽기를 분리해 필요한 컨텍스트만 사용

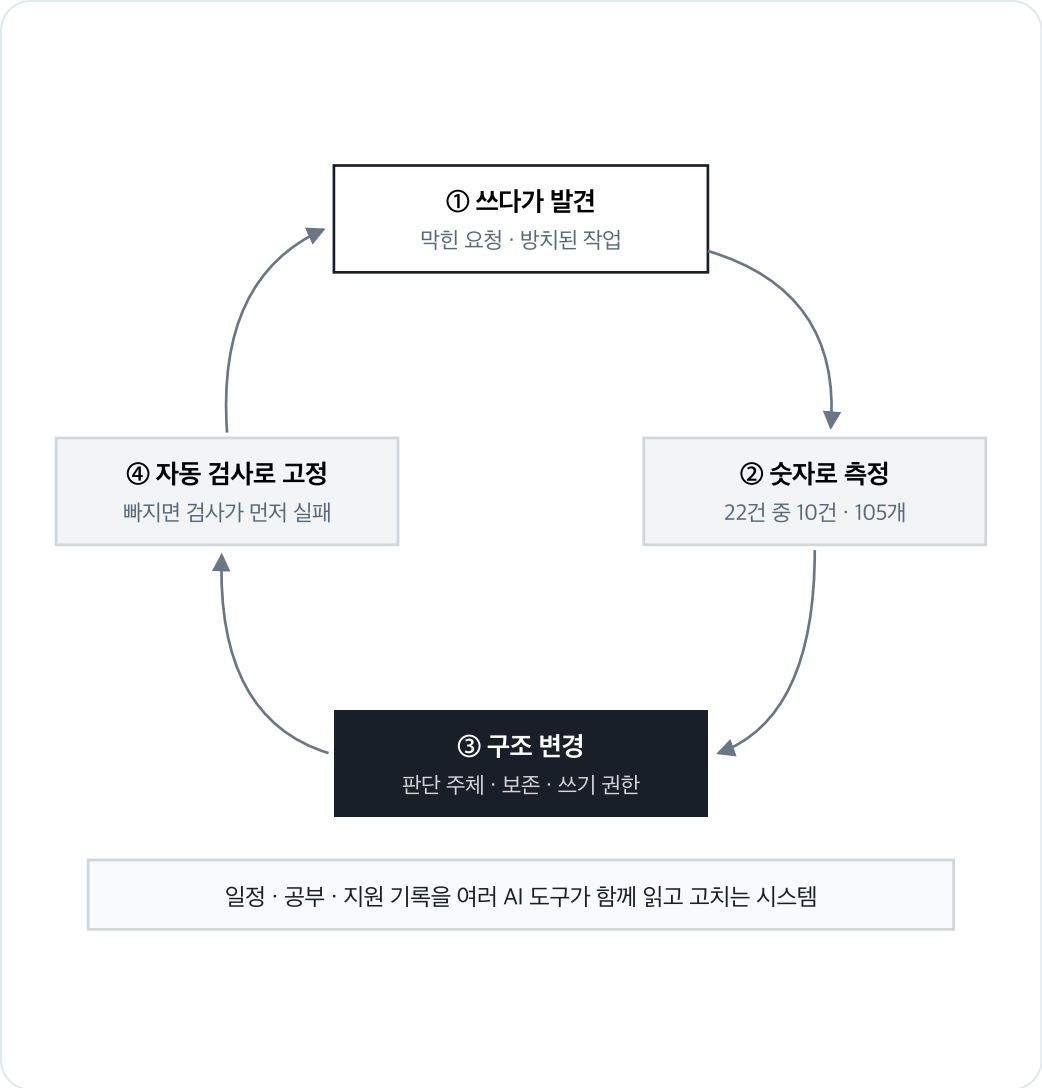
결과

AI 도구가 **먼저 고르고 필요한 원문만 읽는 검색 경로**를 만들었습니다. 문서 처리와 임베딩은 조직 안에서만 돌게 했고, 학부 작품경진대회 은상을 받았습니다.

제 담당 · 검색·조회 API · DB · Docker 배포 · 관리자 화면 (팀 커밋 162개 중 64개)

매일 쓰는 AI 작업 시스템의 막힌 요청과 방치된 작업을 고쳤습니다

문제 해결 8



문제를 숫자로 확인한 뒤 구조를 바꾸는 순서

문제

제 일정·공부·지원 기록을 여러 AI 도구가 함께 다루는 시스템을 직접 설계해 매일 씁니다. 쓰다 보니 음성 요청이 엉뚱한 곳으로 가거나, 여러 작업 창이 동시에 일하며 끝나지 않은 작업이 사라질 위험이 생겼습니다.

요청 분류	음성 요청은 표현이 매번 달라 키워드 규칙이 **22건 중 10건**을 못 찾았습니다 → 규칙을 늘리지 않고, 시스템은 영역 목록과 현재 상태만 주고 의미 판단은 대화 맥락을 아는 AI가 하게 바꿨습니다
작업 보존	끝나지 않은 작업 **105개**가 방치됐고 그중 12개는 어디에도 백업이 없었습니다 → 남의 작업을 대신 저장하지 않고 별도 보존 영역에 시점별로 묶었습니다
변경 통제	기록은 허용된 명령으로만 바꾸고, 그사이 다른 쪽이 고쳤으면 버전 비교로 거부합니다
복구 시험	이력 전체를 파일 하나로 백업하고 실제로 꺼내 봤더니, 흔히 쓰는 복구 명령이 보존 기록을 빠뜨렸습니다 → 복구 절차를 바꿔 6개 문서가 모두 돌아오는 것을 확인했습니다

결과

규칙을 문서로만 두지 않고, 장치가 빠지면 **자동 검사가 먼저 실패**하게 했습니다. 같은 규칙 파일·상태 문서·검사 방식을 주문 자동화 운영에도 그대로 썼습니다.